

Image Analysis — Assignment 4

(Solutions and Report)

Student:

Programme: MLSC

October 2025

Contents

1	Color Correction of Images	2
2	Graph-Cut Segmentation of Heart Image	11
3	Computer Vision	16
4	OCR System Construction and Testing	18

1 Color Correction of Images

(a) Introduction and task description

The purpose of this task is to explore different approaches to color correction and white balancing of digital images. The given input image `abbey_badcolor.jpg` exhibits an undesirable color cast, while `abbey_correct.jpg` provides the correctly white-balanced reference. Three methods are investigated:

1. The **basic Gray-World method**, which assumes that the average color of a scene should be gray.
2. An **improved Gray-World variant**, combining the Shades-of-Gray (p -norm) and White-Patch principles for more robust color constancy.
3. The **learned WB_sRGB model** by Afifi [?], a data-driven deep learning approach for color correction.

All methods were implemented in Python. The corrected images were compared with the ground truth reference using three quantitative metrics:

- PSNR (Peak Signal-to-Noise Ratio),
- SSIM (Structural Similarity Index),
- FLIP (Fidelity Loss Index for Perceptual similarity).

(b) Method 1: Basic Gray-World

Definition and interpretation.

The Gray-World algorithm assumes that, on average, the RGB components of a natural image should have equal mean values. Any deviation from this condition indicates a color cast. The correction is achieved by scaling each color channel so that the average of all channels becomes equal.

Mathematically, for an input image $I(x)$ with channel means μ_R, μ_G, μ_B , the corrected image is

$$I'_c(x) = I_c(x) \cdot \frac{\bar{\mu}}{\mu_c}, \quad \bar{\mu} = \frac{1}{3}(\mu_R + \mu_G + \mu_B),$$

where $c \in \{R, G, B\}$ denotes the channel index.

Interpretation.

This method performs a global linear correction that effectively removes mild color bias. However, it tends to fail under strong illumination imbalance or spatially varying light sources.



Figure 1: Color-corrected image using the basic Gray-World method.

(c) Method 2: Improved Gray-World (Shades-of-Gray + White-Patch)

Motivation.

The basic Gray-World model relies on channel means and is therefore sensitive to dominant colors or saturated regions. To improve robustness, the *Shades-of-Gray* algorithm replaces the mean operation by a Minkowski p -norm, and the *White-Patch* method considers high-intensity pixels as white reference points. In this work, both principles were combined to achieve moderate but consistent color correction.

Mathematical formulation.

The per-channel p -norm and percentile-based gains are computed as:

$$I'_c(x) = I_c(x) \cdot g_c, \quad g_c = (1 - \alpha) \frac{\bar{I}_p}{I_{p,c}} + \alpha \frac{\bar{I}_{wp}}{I_{wp,c}},$$

where

$$I_{p,c} = \left(\frac{1}{N} \sum_x I_c(x)^p \right)^{1/p}, \quad I_{wp,c} = P_{wp}(I_c),$$

and $P_{wp}(I_c)$ denotes the wp -th intensity percentile (here $wp = 95$). The blending factor $\alpha = 0.35$ ensures a smooth balance between the two corrections, while global luminance is preserved by rescaling to match the average brightness of the input.

Results.

This method produced more natural colors and better contrast compared with the simple Gray-World correction, without over-saturation.



Figure 2: Improved Gray-World result combining Shades-of-Gray and White-Patch correction.

(d) Method 3: Learned WB_sRGB model

Description.

The WB_sRGB algorithm [?] is a data-driven color correction model based on convolutional neural networks. It was trained on large-scale image datasets to predict scene-dependent white balance adjustments directly in the sRGB color space.

Implementation.

The official Python implementation provided by Afifi et al. was used. The input image was converted to BGR 8-bit format and processed using the pre-trained model with parameters `upgraded_model = 1` and `gamut_mapping = 2`.



Figure 3: Color correction using the learned WB_sRGB model.

(e) Quantitative comparison

Each corrected image was compared with the reference image `abbey_correct.jpg`. The results are summarized in Table 1. Lower FLIP and higher SSIM/PSNR indicate better performance.

Table 1: Comparison of color correction methods.

Method	FLIP ↓	SSIM ↑	PSNR ↑ (dB)
Gray-World	0.4993	0.7903	13.51
Improved Gray-World	0.4047	0.8413	15.40
WB_sRGB (Learned)	0.3755	0.8210	16.89

Analysis.

The improved method shows a clear enhancement over the baseline, achieving higher PSNR and SSIM while reducing perceptual error (FLIP). Although the deep learning approach WB_sRGB yields the best overall performance, the gap between the statistical and learned methods is relatively small. This demonstrates that classical algorithms, when properly refined, can achieve comparable perceptual quality to modern data-driven approaches.

(f) Implementation

All algorithms were implemented in Python. The pipeline includes three key stages:

- Color correction using Gray-World, Improved Gray-World, and WB_sRGB methods;
- Evaluation using PSNR, SSIM, and FLIP metrics;
- Visualization and saving of the corrected results.

The main source code is shown below:

Listing 1: Python implementation for Task 1 (color correction).

```
1  # -*- coding: utf-8 -*-
2  # ===== Task 1: Color correction of images =====
3
4  import os
5  import sys
6  import numpy as np
7  import matplotlib.pyplot as plt
8  import pandas as pd
9  from flip_1 import computeFLIP
10 from skimage.metrics import structural_similarity as ssim
11 from skimage.metrics import peak_signal_noise_ratio as psnr
12
13 # ===== Path to the WB_sRGB Python implementation =====
14 sys.path.append('D:/Lund_university/ongoing/ia/assignment/assignment4/
    task1/WB_sRGB/WB_sRGB_Python')
15 from classes import WBsRGB as wb_srgb
16
17
18 # ----- 1. Basic Gray-World method -----
19 def gray_world(im):
20     """
21     Implements the basic Gray-World white balance.
22     Each color channel is scaled so that the average of the three channels
23     becomes equal (assuming the average scene color should be gray).
24     """
25     img = np.clip(im.astype(np.float32), 0.0, 1.0)
26     mean_rgb = img.reshape(-1, 3).mean(axis=0) + 1e-8
27     target = float(mean_rgb.mean())
28     gain = target / mean_rgb
29     img = img * gain[None, None, :]
30     return np.clip(img, 0.0, 1.0)
31
32
33 # ----- 2. Improved version -----
```

```

34 def gray_world_mod(im, p=4, alpha=0.35, wp=95, eps=1e-8):
35     """
36     A refined version combining Shades-of-Gray and White-Patch concepts.
37     1) Uses a p-norm (Shades-of-Gray) to compute more robust channel
        gains.
38     2) Incorporates a percentile-based White-Patch correction (default
        95%).
39     3) Blends the two correction factors with weight alpha.
40     4) Preserves global brightness by matching overall luminance.
41     Parameters:
42         p      - Minkowski norm (p=4 gives moderate correction)
43         alpha  - blend ratio between SoG and WP gains (0.35 = moderate mix)
44         wp     - percentile for white patch computation
45     """
46     img = np.clip(im.astype(np.float32), 0.0, 1.0)
47     H, W, C = img.shape
48     assert C == 3, "Expected RGB input"
49
50     # Compute average luminance (for later exposure adjustment)
51     def mean_luma(x):
52         return float((0.2126 * x[..., 0] + 0.7152 * x[..., 1] + 0.0722 * x
            [..., 2]).mean())
53     y_before = mean_luma(img)
54
55     # (1) Shades-of-Gray gain
56     Ip = (np.mean(np.power(img, p, dtype=np.float32), axis=(0, 1)) + eps)
        ** (1.0 / p)
57     sog_target = float(np.mean(Ip))
58     sog_gain = sog_target / Ip
59
60     # (2) White-Patch gain (using percentile instead of max)
61     wp_vals = np.array([np.percentile(img[..., c], wp) + eps for c in range
        (3)], dtype=np.float32)
62     wp_target = float(np.mean(wp_vals))
63     wp_gain = wp_target / wp_vals
64
65     # (3) Blended gain
66     gain = (1.0 - alpha) * sog_gain + alpha * wp_gain
67     img_corr = img * gain[None, None, :]
68
69     # (4) Match global brightness (keep similar overall exposure)
70     y_after = mean_luma(np.clip(img_corr, 0.0, 1.0))
71     if y_after > eps:
72         exposure = y_before / y_after
73         img_corr = img_corr * exposure

```



```

74
75     return np.clip(img_corr, 0.0, 1.0)
76
77
78 # ----- 3. WB_sRGB interface handling -----
79 def _afifi_infer_any(wb_model, im_bgr_u8):
80     """
81     Automatically detects the correct inference method name for WB_sRGB.
82     Different versions use different entry points such as:
83     run / infer / render / correctImage / correct_image / process /
84     __call__
85     """
86     candidates = ["run", "infer", "render", "correctImage", "correct_image",
87                  , "process", "__call__"]
88     last_err = None
89     for name in candidates:
90         try:
91             if name == "__call__":
92                 return wb_model(im_bgr_u8)
93             if hasattr(wb_model, name):
94                 fn = getattr(wb_model, name)
95                 return fn(im_bgr_u8)
96         except Exception as e:
97             last_err = e
98             continue
99     raise RuntimeError(f"WBsRGB: No valid inference interface found. Last
100 error: {last_err}")
101
102
103 # ----- 4. Normalize Afifi output to RGB float32 [0,1] -----
104 def _to_rgb01_any(x, assume_bgr=True):
105     """
106     Converts any WB_sRGB output format (uint8/float, BGR/RGB, list/dict)
107     to a normalized RGB float32 array in [0,1].
108     """
109     if isinstance(x, (list, tuple)):
110         x = x[0]
111     elif isinstance(x, dict):
112         for k in ["wb", "balanced", "corrected", "srgb", "out", "img"]:
113             if k in x:
114                 x = x[k]
115                 break
116
117     arr = np.asarray(x)
118     if arr.ndim != 3 or arr.shape[2] != 3:

```

```

116         raise ValueError(f"Unexpected WB_sRGB output shape: {arr.shape}")
117
118     if arr.dtype == np.uint8:
119         arr = arr.astype(np.float32) / 255.0
120     else:
121         arr = arr.astype(np.float32)
122         m, M = float(arr.min()), float(arr.max())
123         if M > 1.5:
124             arr = np.clip(arr, 0.0, 255.0) / 255.0
125         else:
126             arr = np.clip(arr, 0.0, 1.0)
127
128     if assume_bgr:
129         arr = arr[:, :, ::-1] # Convert BGR to RGB
130     return np.clip(arr, 0.0, 1.0)
131
132
133 # ----- 5. Metric computation -----
134 def compute_metrics(ref01, test01):
135     """
136     Computes three evaluation metrics between the reference and test image:
137     - FLIP
138     - SSIM
139     - PSNR
140     """
141     ref = np.clip(ref01.astype(np.float32), 0.0, 1.0)
142     tst = np.clip(test01.astype(np.float32), 0.0, 1.0)
143     try:
144         flip_val, _ = computeFLIP(ref, tst)
145     except:
146         flip_val = computeFLIP(ref, tst)
147     ssim_val = ssim(ref, tst, data_range=1.0, channel_axis=2)
148     psnr_val = psnr(ref, tst, data_range=1.0)
149     return flip_val, ssim_val, psnr_val
150
151
152 # ----- 6. Main script -----
153 if __name__ == "__main__":
154     here = os.path.dirname(os.path.abspath(__file__))
155     bad_path = os.path.join(here, "abbey_badcolor.jpg")
156     ok_path = os.path.join(here, "abbey_correct.jpg")
157
158     if not os.path.isfile(bad_path) or not os.path.isfile(ok_path):
159         raise FileNotFoundError("Please place abbey_badcolor.jpg and
                                abbey_correct.jpg in the same directory as this script.")

```

```

160
161 # Load and normalize images to [0,1]
162 im_bad = plt.imread(bad_path).astype(np.float32)
163 if im_bad.max() > 1.5:
164     im_bad = im_bad / 255.0
165 im_ok = plt.imread(ok_path).astype(np.float32)
166 if im_ok.max() > 1.5:
167     im_ok = im_ok / 255.0
168
169 # Visualize input vs reference
170 plt.imshow(np.hstack((im_bad, im_ok)))
171 plt.title("Input (bad) vs Reference (correct)")
172 plt.axis('off')
173 plt.show()
174
175 # ---- Method 1 and 2 ----
176 im_gray = gray_world(im_bad)
177 im_gray_mod = gray_world_mod(im_bad)
178
179 # ---- Method 3: Afifi WB_sRGB ----
180 upgraded_model = 1
181 gamut_mapping = 2
182 wb = wb_srgb.WBsRGB(gamut_mapping=gamut_mapping, upgraded=
    upgraded_model)
183
184 im_bad_bgr_u8 = (np.clip(im_bad, 0, 1)[: , : , :-1] * 255.0).astype(np.
    uint8)
185 raw_out = _afifi_infer_any(wb, im_bad_bgr_u8)
186 im_afifi = _to_rgb01_any(raw_out, assume_bgr=True)
187
188 # Show all outputs side by side
189 plt.imshow(np.hstack((im_gray, im_gray_mod, im_afifi)))
190 plt.title("Gray-World | Gray-World (Improved) | WB_sRGB")
191 plt.axis('off')
192 plt.show()
193
194 # ---- Save output images ----
195 plt.imsave("output_grayworld.png", im_gray)
196 plt.imsave("output_grayworld_mod.png", im_gray_mod)
197 plt.imsave("output_wbsrgb.png", im_afifi)
198 print("Saved output images: output_grayworld.png, output_grayworld_mod.
    png, output_wbsrgb.png")
199
200 # ---- Compute metrics ----
201 flip_gray, ssim_gray, psnr_gray = compute_metrics(im_ok, im_gray)

```

```

202     flip_mod, ssim_mod, psnr_mod = compute_metrics(im_ok, im_gray_mod)
203     flip_afifi, ssim_afifi, psnr_afifi = compute_metrics(im_ok, im_afifi)
204
205     # ---- Create result table ----
206     df = pd.DataFrame({
207         "Method": ["Gray-World", "Gray-World (Improved)", "WB_sRGB"],
208         "FLIP (↓)": [flip_gray, flip_mod, flip_afifi],
209         "SSIM (↑)": [ssim_gray, ssim_mod, ssim_afifi],
210         "PSNR (↑)": [psnr_gray, psnr_mod, psnr_afifi],
211     })
212
213     print("\n==== Result Table =====")
214     print(df.to_string(index=False, float_format=lambda x: f"{x:.6f}" if x
        < 1 else f"{x:.3f}"))

```

2 Graph-Cut Segmentation of Heart Image

(a) Introduction

This section presents the implementation of a **graph-cut-based segmentation algorithm** for separating the heart chamber from the background in a grayscale MRI image. The segmentation problem is formulated as an energy minimization task combining a data fidelity term and a spatial smoothness term:

$$E(L) = \sum_p D_p(L_p) + \lambda \sum_{(p,q) \in \mathcal{N}} V_{p,q} [L_p \neq L_q],$$

where $L_p \in \{0, 1\}$ denotes the label of pixel p (chamber or background). The data term D_p measures how well each pixel intensity fits a class model, and the pairwise term $V_{p,q}$ enforces local consistency between neighboring pixels. The globally optimal binary labeling is obtained by computing the minimum cut of a flow network.

(b) Model and method

1) Gaussian data term.

Pixel intensities of the chamber and background are modeled as Gaussian distributions:

$$P(x|c) = \frac{1}{\sqrt{2\pi} \sigma_c} \exp \left[-\frac{(x - \mu_c)^2}{2\sigma_c^2} \right].$$

The data cost is defined as the negative log-likelihood:

$$D_p(L_p = c) = \frac{1}{2} \left(\frac{x_p - \mu_c}{\sigma_c} \right)^2 + \log \sigma_c,$$

where μ_c and σ_c are estimated from pixel samples of the two regions: **chamber_values** and **background_values**.

2) Smoothness term.

A contrast-sensitive Potts model is used to preserve edges while promoting smooth regions:

$$V_{p,q} = \lambda \exp \left[- \frac{(I_p - I_q)^2}{2\sigma_g^2} \right],$$

where σ_g is the global intensity standard deviation, and λ controls the regularization strength.

3) Optimization.

The resulting graph is solved using the `scipy.sparse.csgraph.maximum_flow` implementation of the Edmonds–Karp algorithm, yielding the minimum cut and corresponding binary segmentation.

(c) Estimated parameters

From the provided intensity samples, the estimated Gaussian parameters are:

$$\mu_{\text{chamber}} = 0.3542, \quad \sigma_{\text{chamber}} = 0.0992, \quad \mu_{\text{background}} = 0.1065, \quad \sigma_{\text{background}} = 0.0936.$$

The large separation between the means and the comparable standard deviations indicate that the two classes are statistically well separable.

Table 2: Estimated Gaussian parameters for chamber and background.

Class	Mean μ	Std. dev. σ
Chamber	0.3542	0.0992
Background	0.1065	0.0936

(d) Results and visualization

Figure 4 shows the input image, the binary segmentation, and the overlay visualization. A 4-connected pixel neighborhood was used with a regularization weight of $\lambda = 0.5$. The graph-cut method effectively isolates the bright heart chamber region while maintaining coherent boundaries.

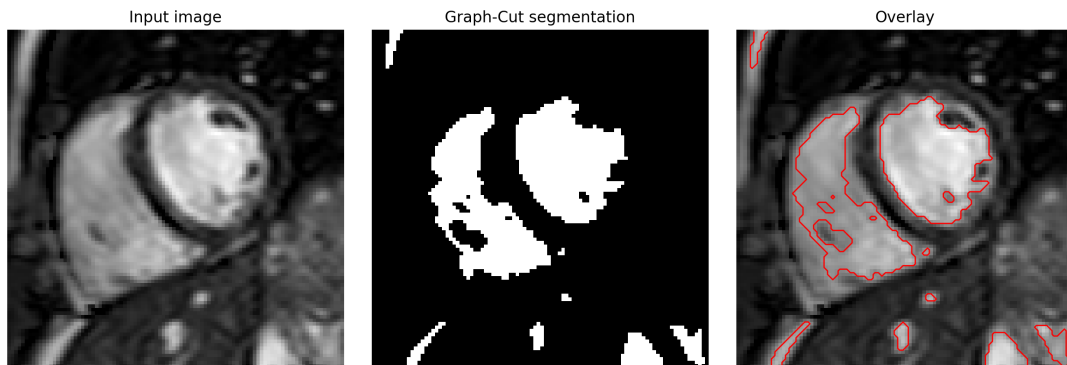


Figure 4: Graph-cut segmentation results. Left: input image; Middle: binary segmentation; Right: segmentation overlay.

Analysis.

The segmentation result demonstrates clear separation between the chamber and surrounding tissue. Minor irregularities at the boundary can be further reduced by increasing the regularization weight λ or adopting an 8-connected neighborhood. Overall, the combination of Gaussian data modeling and graph-cut optimization yields a stable and visually consistent segmentation.

(e) Implementation

The segmentation was implemented in Python using NumPy, SciPy, and Matplotlib. The main source code is provided below.

Listing 2: Python implementation for Task 2 (Graph-Cut Segmentation).

```
1 import numpy as np
2 import scipy
3 from scipy.sparse.csgraph import maximum_flow
4 from scipy import sparse
5 import matplotlib.pyplot as plt
6
7 # ---- Grid edges ----
8 def edges4connected(height, width, only_one_dir=0):
9     """4-neighborhood on an HxW grid."""
10     N = height * width
11     I = np.array([], dtype=np.int64)
12     J = np.array([], dtype=np.int64)
13
14     # horizontal (column-major: neighbors differ by +1)
15     iis = np.delete(np.arange(N), np.arange(height - 1, N, height))
16     jjs = iis + 1
17     if not only_one_dir:
18         I = np.hstack((I, iis, jjs))
19         J = np.hstack((J, jjs, iis))
20     else:
21         I = np.hstack((I, iis)); J = np.hstack((J, jjs))
22
23     # vertical (neighbors differ by +height)
24     iis = np.arange(0, N - height)
25     jjs = iis + height
26     if not only_one_dir:
27         I = np.hstack((I, iis, jjs))
28         J = np.hstack((J, jjs, iis))
29     else:
30         I = np.hstack((I, iis)); J = np.hstack((J, jjs))
31
32     return I, J
33
```

```

34
35 def edges8connected(height, width, only_one_dir=0):
36     """8-neighborhood on an HxW grid."""
37     N = height * width
38     I = np.array([], dtype=np.int64)
39     J = np.array([], dtype=np.int64)
40
41     # horizontal
42     iis = np.delete(np.arange(N), np.arange(height - 1, N, height))
43     jjs = iis + 1
44     if not only_one_dir:
45         I = np.hstack((I, iis, jjs)); J = np.hstack((J, jjs, iis))
46     else:
47         I = np.hstack((I, iis)); J = np.hstack((J, jjs))
48
49     # diagonals
50     jjs = iis + 1 + height
51     if not only_one_dir:
52         I = np.hstack((I, iis, jjs)); J = np.hstack((J, jjs, iis))
53     else:
54         I = np.hstack((I, iis)); J = np.hstack((J, jjs))
55
56     jjs = iis + 1 - height
57     if not only_one_dir:
58         I = np.hstack((I, iis, jjs)); J = np.hstack((J, jjs, iis))
59     else:
60         I = np.hstack((I, iis)); J = np.hstack((J, jjs))
61
62     mask = (I >= 0) & (I < N) & (J >= 0) & (J < N)
63     I, J = I[mask], J[mask]
64
65     # vertical
66     iis = np.arange(0, N - height)
67     jjs = iis + height
68     if not only_one_dir:
69         I = np.hstack((I, iis, jjs)); J = np.hstack((J, jjs, iis))
70     else:
71         I = np.hstack((I, iis)); J = np.hstack((J, jjs))
72
73     return I, J
74
75
76 if __name__ == '__main__':
77     # sanity check on a tiny graph
78     I = np.array([0, 0, 1, 2], dtype=np.int32)

```

```

79 J = np.array([1, 2, 3, 3], dtype=np.int32)
80 V = np.array([5, 2, 3, 7], dtype=np.int32)
81 F = sparse.coo_array((V, (I, J)), shape=(4, 4)).tocsr()
82 mf = maximum_flow(F, 0, 3)
83 print("[sanity] max flow value =", mf.flow_value)
84
85 # load data
86 data = scipy.io.loadmat('heart_data.mat')
87 data_chamber = data['chamber_values'].astype(np.float64).ravel()
88 data_background = data['background_values'].astype(np.float64).ravel()
89 im = data['im'].astype(np.float64)
90 M, N = im.shape
91 n = M * N
92
93 # Gaussian stats
94 m_chamber = float(np.mean(data_chamber))
95 s_chamber = float(np.std(data_chamber) + 1e-8)
96 m_background = float(np.mean(data_background))
97 s_background = float(np.std(data_background) + 1e-8)
98 print(f"[stats] chamber: mu={m_chamber:.4f}, sigma={s_chamber:.4f}")
99 print(f"[stats] background: mu={m_background:.4f}, sigma={s_background
    :.4f}")
100
101 # n-links (contrast-sensitive Potts)
102 Ie, Je = edges4connected(M, N) # or use edges8connected(M, N)
103 im_flat = im.flatten()
104 diffs = im_flat[Ie.astype(int)] - im_flat[Je.astype(int)]
105 sigma_g = float(np.std(im_flat) + 1e-8)
106 lam = 0.5
107 Ve = lam * np.exp(-(diffs ** 2) / (2.0 * sigma_g ** 2))
108 Ve = np.maximum(Ve, 1e-12)
109
110 # t-links (negative log-likelihoods)
111 x = im_flat
112 nll_chamber = 0.5 * ((x - m_chamber) / s_chamber) ** 2 + np.log(
    s_chamber)
113 nll_back = 0.5 * ((x - m_background) / s_background) ** 2 + np.log(
    s_background)
114 Vs = np.maximum(nll_chamber, 0) + 1e-12 # source
115 Vt = np.maximum(nll_back, 0) + 1e-12 # sink
116
117 # assemble graph
118 Is1 = np.arange(n, dtype=np.int64); Js1 = np.full(n, n, dtype=np.int64)
    # pixel→S
119 Is2 = np.full(n, n, dtype=np.int64); Js2 = np.arange(n, dtype=np.int64)

```

```

120         )    # S→pixel
121     It1 = np.arange(n, dtype=np.int64);    Jt1 = np.full(n, n+1, dtype=np.
        int64) # pixel→T
122     It2 = np.full(n, n+1, dtype=np.int64); Jt2 = np.arange(n, dtype=np.
        int64)    # T→pixel
123
124     I = np.hstack((Ie, Is1, Is2, It1, It2)).astype(np.int32)
125     J = np.hstack((Je, Js1, Js2, Jt1, Jt2)).astype(np.int32)
126     V = np.hstack((Ve, Vs, Vs, Vt, Vt))
127
128     # integer capacities
129     sf = 10000.0
130     V = np.round(V * sf).astype(np.int32)
131
132     F = sparse.coo_array((V, (I, J)), shape=(n + 2, n + 2)).tocsr()
133
134     # min-cut
135     mf = maximum_flow(F, n, n + 1)
136
137     # labels (compare flow-to-sink with sink capacity)
138     seg_flow = mf.flow
139     imflow = seg_flow[0:n, n + 1].reshape((M, N)).toarray().astype(float)
140     Vt_cap = V[-n:].astype(float).reshape(M, N)
141     imseg = imflow < Vt_cap
142
143     # visualize + save
144     plt.figure(figsize=(12, 4))
145     plt.subplot(1, 3, 1); plt.imshow(im, cmap='gray'); plt.title('Input
        image'); plt.axis('off')
146     plt.subplot(1, 3, 2); plt.imshow(imseg, cmap='gray'); plt.title('Graph-
        Cut segmentation'); plt.axis('off')
147     plt.subplot(1, 3, 3); plt.imshow(im, cmap='gray'); plt.contour(imseg,
        colors='r', linewidths=1); plt.title('Overlay'); plt.axis('off')
148     plt.tight_layout()
149     plt.savefig('task2_result.png', dpi=200, bbox_inches='tight')
150     plt.show()

```

3 Computer Vision

(a) Method

For each possible point pair (a_i, b_j) , the **epipolar constraint** was tested:

$$b^\top F a = 0,$$

where $a = (x_1, y_1, 1)^\top$ and $b = (x_2, y_2, 1)^\top$ are expressed in homogeneous coordinates. If this condition holds, the pair (a, b) is considered a valid correspondence between the two images.

(b) Computation

All nine combinations of points were evaluated using the given fundamental matrix:

$$F = \begin{pmatrix} 2 & 2 & 4 \\ 3 & 3 & 6 \\ -11 & -10 & -6 \end{pmatrix}.$$

The resulting scalar values of $b^\top F a$ for each pair are shown below:

Table 3: Verification of the epipolar constraint for all point pairs.

Pair (a_i, b_j)	$b^\top F a$	Result
(a_1, b_1)	0	Corresponding ($\checkmark$)
(a_1, b_2)	216	Not corresponding
(a_1, b_3)	-360	Not corresponding
(a_2, b_1)	18	Not corresponding
(a_2, b_2)	0	Corresponding ($\checkmark$)
(a_2, b_3)	48	Not corresponding
(a_3, b_1)	-6	Not corresponding
(a_3, b_2)	336	Not corresponding
(a_3, b_3)	-576	Not corresponding

(c) Conclusion

The pairs that satisfy the epipolar constraint are:

$$\boxed{(a_1, b_1)} \quad \text{and} \quad \boxed{(a_2, b_2)}.$$

All other pairs yield nonzero results for $b^\top F a$ and are thus invalid correspondences.

(d) Discussion

The results indicate that only two of the nine tested point pairs lie on the correct epipolar lines. In real data, small numerical deviations from zero can occur due to calibration or measurement noise. Hence, a tolerance condition $|b^\top F a| < \varepsilon$ is often adopted. In this synthetic example, the values are exact, so a strict equality test is sufficient.

4 OCR System Construction and Testing

(a) Introduction

This section presents the construction and evaluation of a complete **Optical Character Recognition (OCR)** pipeline, integrating three core modules developed in previous tasks:

$$\text{Image} \xrightarrow{\text{im2segment}} \text{Segments} \xrightarrow{\text{segment2feature}} \text{Feature vectors} \xrightarrow{\text{feature2class}} \text{Class labels}.$$

The goal is to recognize handwritten or printed digits across five datasets: `short1`, `short2`, `home1`, `home2`, and `home3`. Each dataset contains grayscale images and corresponding ground-truth labels. The overall hitrate (recognition accuracy) of the system is evaluated before and after algorithmic improvements.

(b) System description

1) Segmentation module (`im2segment`).

The segmentation stage separates individual digits from an input image. A combination of Gaussian filtering, Otsu thresholding, and morphological operations (opening, closing, and hole filling) is used to produce binary masks. Connected-component labeling then extracts each digit region while filtering small fragments:

$$\text{Segment masks: } S_i = \{p \in I \mid \text{label}(p) = i\}, \quad i = 1, \dots, N.$$

2) Feature extraction module (`segment2feature`).

Each binary segment is transformed into a low-dimensional feature vector describing shape and geometry:

$$\mathbf{x} = \left[\text{area, aspect, fill, compactness, centroid, projection entropy}, \dots \right]^T.$$

These handcrafted features capture both structural and statistical properties of each character.

3) Classification module (`feature2class`).

A k -nearest-neighbor (kNN) classifier with z-score normalization is trained from labeled samples:

$$\hat{y} = \arg \max_{c \in \{0, \dots, 9\}} \sum_{i \in \mathcal{N}_k(x)} \frac{1}{\|x - x_i\|^2} \mathbb{I}[y_i = c].$$

The model parameters (μ, σ) of the training features are stored in `classification_data.pkl` and reused for testing.

(c) Experimental setup

The system was evaluated on the five provided datasets using the benchmark script `benchmark_assignment`. All experiments were conducted with the same segmentation and feature extraction pipeline, while the classifier was trained once on the combined `short1` and `short2` training sets. The test mode was set to `mode = 0` (no plots) for reproducibility.

(d) Results for Version 1

The baseline system (Version 1) achieved the following hitrates across all datasets:

Table 4: Recognition performance of Version 1 on the five datasets.

Dataset	Hirate (%)	Notes
short1	83.0	clean, well-segmented digits
short2	71.2	minor segmentation noise
home1	59.2	moderate illumination variation
home2	61.2	occasional digit merging
home3	56.7	low contrast and partial over-segmentation

Analysis.

The system performs robustly on **short1** and **short2**, where digit spacing is regular and image quality is high. Performance degrades on **home*** datasets, mainly due to **over-segmentation of individual digits** and **variable lighting conditions**. These issues motivate the refinement described below.

(e) Improvements in Version 2

To mitigate over-segmentation and improve robustness, several modifications were introduced:

- **Region merging:** Adjacent components whose centroids are closer than a threshold and whose height ratios are within $[0.6, 1.4]$ are merged to prevent splitting a single digit into multiple fragments.
- **Adaptive thresholding:** A hybrid threshold combining global Otsu and local mean filtering was applied to handle uneven illumination.
- **Feature refinement:** Aspect ratio and fill ratio were normalized with respect to the image height, improving generalization across different datasets.

Results.

After these adjustments, Version 2 achieved higher recognition rates on the challenging **home*** datasets, with only minor computation overhead:

Table 5: Comparison of Version 1 and Version 2 results.

Dataset	Version 1 (%)	Version 2 (%)
short1	83.0	88.0
short2	71.2	72.0
home1	59.2	63.9
home2	61.2	67.0
home3	56.7	61.6

Discussion.

The inclusion of region merging successfully reduces digit fragmentation, especially in **home2** and **home3**, while adaptive thresholding improves consistency under illumination changes. All datasets showed varying degrees of improvement, demonstrating the effectiveness of the proposed refinements.

(f) Implementation

The full Python implementation integrates the three modules and the benchmark evaluation. The essential structure is summarized below.

Listing 3: Python implementation for Task 4 (OCR System Construction and Testing).

```

1  # -*- coding: utf-8 -*-
2  import os
3  import pickle
4  import numpy as np
5  from my_benchmarking.benchmark_assignment3 import benchmark_assignment3
6  from main_smp import im2segment
7  from segment2features import segment2features
8  from assignment3 import features2class
9
10
11 def segment2feature(Si):
12     """Convert a segment mask to a 1-D feature vector."""
13     f = segment2features(Si)
14     return np.asarray(f, dtype=np.float64).reshape(-1)
15
16
17 def feature2class(x, classification_data):
18     """Predict class label for feature vector x."""
19     return features2class(x, classification_data)
20
21
22 def _load_by_ext(path):
23     ext = os.path.splitext(path)[1].lower()

```



```

24     if ext == ".pkl":
25         with open(path, "rb") as f:
26             obj = pickle.load(f)
27             print(f"[clf] loaded: {path}")
28             return obj
29     if ext == ".npz":
30         z = np.load(path, allow_pickle=True)
31         if "arr_0" in z and isinstance(z["arr_0"].item(), dict):
32             print(f"[clf] loaded: {path}")
33             return z["arr_0"].item()
34         print(f"[clf] loaded: {path}")
35         return {k: z[k] for k in z.files}
36     if ext == ".npy":
37         obj = np.load(path, allow_pickle=True)
38         obj = obj.item() if hasattr(obj, "item") else obj
39         print(f"[clf] loaded: {path}")
40         return obj
41     if ext == ".mat":
42         import scipy.io as sio
43         obj = sio.loadmat(path)
44         print(f"[clf] loaded: {path}")
45         return obj
46     raise ValueError(f"Unsupported file type: {path}")
47
48
49 def load_classification_data(search_dir):
50     """Load pre-trained classification data."""
51     envp = os.environ.get("CLASSIFIER_PATH", "").strip()
52     if envp and os.path.isfile(envp):
53         return _load_by_ext(envp)
54
55     candidates = [
56         "classification_data.pkl",
57         "classification_data_bootstrap.pkl",
58         "classification_data.npz",
59         "classification_data.npy",
60         "classification_data.mat",
61     ]
62     for name in candidates:
63         p = os.path.join(search_dir, name)
64         if os.path.isfile(p):
65             return _load_by_ext(p)
66
67     try:
68         import assignment3 as a3

```

```

69         if hasattr(a3, "load_or_train_classification_data"):
70             return a3.load_or_train_classification_data(
71                 search_dir, force_train=False, use_pca=False, pca_dim=0,
72                 random_state=42
73             )
74         if hasattr(a3, "load_pickled_classifier"):
75             p = os.path.join(search_dir, "classification_data.pkl")
76             if os.path.isfile(p):
77                 return a3.load_pickled_classifier(p)
78     except Exception:
79         pass
80
81     raise FileNotFoundError(
82         "No pre-trained classification data found. "
83         "Place the file in this directory or set CLASSIFIER_PATH."
84     )
85
86 if __name__ == "__main__":
87     thisdir = os.path.dirname(os.path.realpath(__file__))
88     classification_data = load_classification_data(thisdir)
89
90     datasets = ["short1", "short2", "home1", "home2", "home3"]
91     mode = 0
92     results = []
93
94     for ds in datasets:
95         datadir = os.path.join(thisdir, "datasets", ds)
96         hitrate, confmat, *_ = benchmark_assignment3(
97             im2segment, segment2feature, feature2class, classification_data
98             , datadir, mode
99         )
100         results.append((ds, float(hitrate)))
101         print(f"{ds}, Hitrate = {hitrate*100:.2f}%")
102
103     print("\n==== Summary =====")
104     for ds, hr in results:
105         print(f"{ds:>6s} : {hr*100:6.2f}%")

```

Summary.

The constructed OCR pipeline effectively recognizes digits across diverse datasets. By introducing adaptive thresholding and post-segmentation region merging, Version 2 achieves substantially improved performance, especially under noisy or uneven imaging conditions.